بِسْمِ اللَّهِ الرَّحْمَٰنِ الرَّحِيمِ

In the name of Allah, the most gracious, the most merciful

1

Lec 12: Node.js

Dr. Nida Adnan

Introduction

- Node.js is a JavaScript runtime built on Chrome's V8 JavaScript engine.
- It is an open-source, cross-platform, asynchronous, event-driven runtime designed for building scalable network applications.
- Unlike traditional JavaScript, which runs in the browser, Node.js allows developers to run JavaScript on the server-side.
- It is not advisable to use Node.js for CPU intensive applications.

Key Features of Node.js

- **Asynchronous and Event-Driven:** All API calls are non-blocking, meaning Node.js doesn't wait for a response from an operation (like reading from a file or querying a database) before proceeding to the next task.
- **Single-Threaded:** Node.js operates on a single thread using an event loop, allowing it to handle multiple connections concurrently without the overhead of multiple threads.
- **Cross-Platform:** It can run on various platforms, including Linux, macOS, and Windows.
- **Fast and Scalable:** Due to its event-driven architecture and V8 engine, Node.js can handle many concurrent requests with low latency.

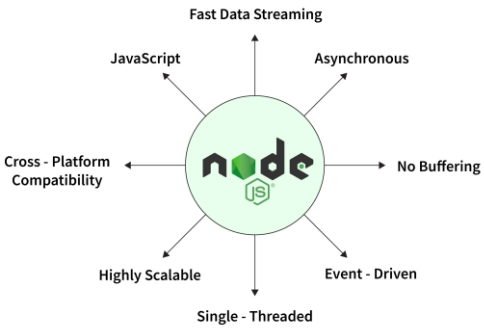
How PHP or ASP handle file request?

1. Sends the task to the computer's file system.
2. Waits while the file system opens and reads the file.
3. Returns the content to the client.
4. Ready to handle the next request.

A common task for a web server can be to open a file on the server and return the content to the client.

How node.js handle file request?

1. Sends the task to the computer's file system.
2. Ready to handle the next request.
3. When the file system has opened and read the file, the server returns the content to the client.



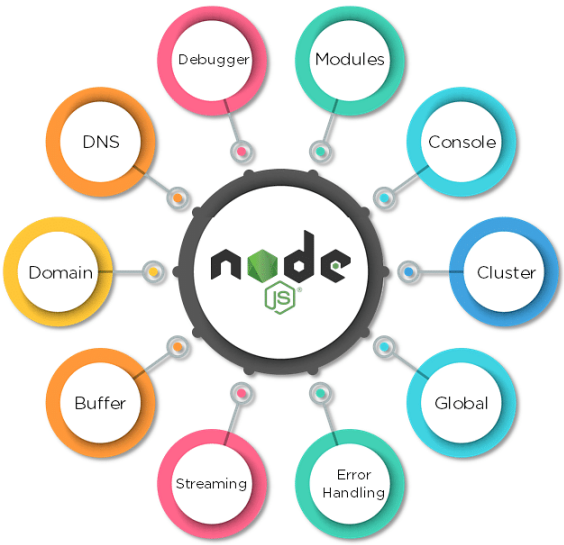
A common task for a web server can be to open a file on the server and return the content to the client.

Node.js eliminates the waiting, and simply continues with the next request.

Node.js runs single-threaded, non-blocking, asynchronous programming, which is very memory efficient.

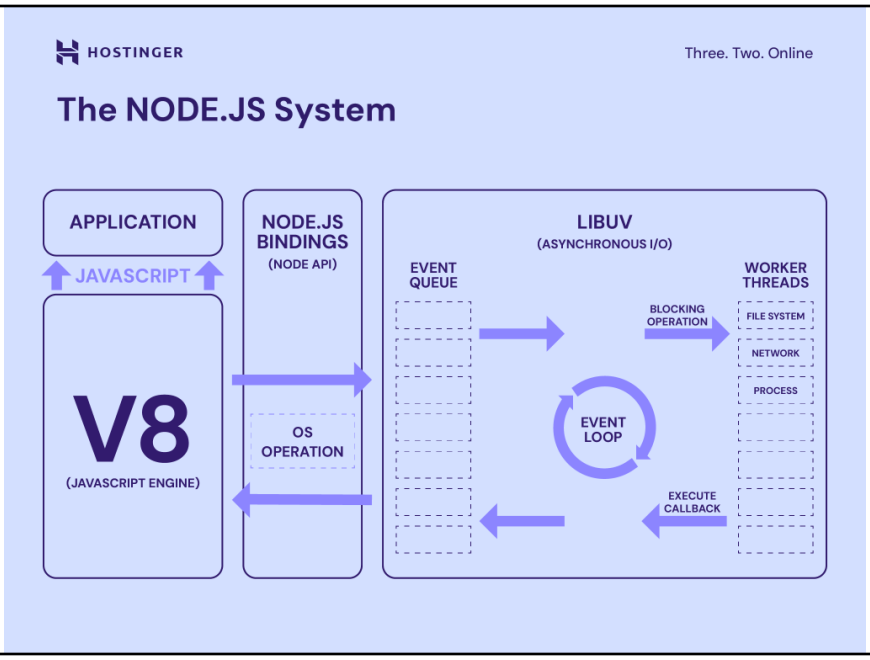
Functions of Node.js

- Node.js can generate dynamic page content
- Node.js can create, open, read, write, delete, and close files on the server
- Node.js can collect form data
- Node.js can add, delete, modify data in your database



What is a Node.js file?

- Node.js files contain tasks that will be executed on certain events
- A typical event is someone trying to access a port on the server
- Node.js files must be initiated on the server before having any effect
- Node.js files have extension ".js"


HOSTINGER
Three. Two. Online
The NODE.JS System
APPLICATION
JAVASCRIPT
V8
(JAVASCRIPT ENGINE)
NODE.JS BINDINGS
(NODE API)
OS OPERATION
LIBUV
(ASYNCHRONOUS I/O)
EVENT QUEUE
WORKER THREADS
BLOCKING OPERATION
FILE SYSTEM
NETWORK
PROCESS
EVENT LOOP
EXECUTE CALLBACK

What is LIBUV?

- libuv is a multi-platform asynchronous I/O library that Node.js uses to handle non-blocking operations.
- It is a core part of the Node.js runtime environment and provides the foundational building blocks for Node.js to perform tasks like file system access, networking, and handling concurrency in an efficient, event-driven way.
- libuv abstracts away the differences in I/O operations across different operating systems (such as Linux, Windows, macOS) and offers a consistent interface to work with, enabling developers to write platform-independent code.
- Its primary focus is on providing asynchronous I/O with a non-blocking event-driven model, which allows Node.js to perform multiple tasks concurrently without waiting for any operation to complete before moving to the next one.

libuv is a multi-platform **asynchronous I/O library** that Node.js uses to handle non-blocking operations. It is a core part of the Node.js runtime environment and provides the foundational building blocks for Node.js to perform tasks like file system access, networking, and handling concurrency in an efficient, event-driven way.
libuv abstracts away the differences in I/O operations across different operating systems (such as Linux, Windows, macOS) and offers a consistent interface to work with, enabling developers to write platform-independent code. Its primary focus is on providing asynchronous I/O with a **non-blocking event-driven model**, which allows Node.js to perform multiple tasks concurrently without waiting for any operation to complete before moving to the next one.

Key Features of libuv:

1.Event Loop:

The **event loop** is the heart of Node.js's asynchronous model, and libuv manages it. The event loop runs continuously, processing events and executing callbacks when certain events (like I/O operations) are ready. This allows Node.js to handle many operations concurrently, even though it runs on a single thread.

2. **Non-blocking I/O**:
   libuv handles I/O operations in a non-blocking way. For example, if you read a file, libuv allows other tasks (like network requests) to proceed while it waits for the file to be read, instead of blocking the entire thread until the file is available.
3. **Cross-platform Compatibility**:
   libuv abstracts system calls, making Node.js applications cross-platform. It offers a consistent API for performing file I/O, networking, and other low-level operations on different operating systems.
4. **Thread Pool**:
   libuv uses a thread pool to perform certain I/O operations in parallel. These operations include file system access, DNS resolution, and certain types of networking. The thread pool allows these operations to occur concurrently, improving performance.
5. **TCP and UDP Support**:
   libuv includes support for both **TCP** and **UDP** protocols, allowing you to create network servers and clients with ease. It provides an abstraction for networking tasks such as establishing a server, listening for connections, and sending/receiving data over sockets.
6. **Timers**:
   libuv manages timers and intervals, enabling non-blocking sleep, setTimeout, and setInterval functionality in a way that doesn't block the event loop.
7. **File System Operations**:
   libuv provides async methods for reading and writing files, making it an essential tool for Node.js to perform file I/O in a non-blocking manner.

How libuv Works in Node.js

Node.js is single-threaded, but it can handle multiple operations concurrently through **libuv**. This is done through a non-blocking event-driven architecture. When an I/O operation (like reading a file or making a network request) is initiated, Node.js delegates the operation to libuv, which manages the operation asynchronously.

•When an I/O task is started (e.g., reading from a file), Node.js uses libuv to delegate the task to the underlying operating system's system calls or to a thread pool.

•While the I/O operation is in progress, the event loop continues to run, processing other events and tasks.

•Once the I/O operation finishes, libuv notifies Node.js, which then executes the callback associated with that I/O task, delivering the result.

This non-blocking, asynchronous model ensures that Node.js can handle thousands of concurrent connections without the need for multiple threads.

Install Node.js

Install Node.js: Visit the Node.js website and download the latest stable version for your operating system.

- Windows and macOS: Download the installer and follow the prompts.
- Linux: Use a package manager like apt, yum, or dnf to install Node.js.

Once installed, verify the installation using the following commands:

```
node –v
npm -v
```

This should display the Node.js version and the Node Package Manager (npm) version.

Setting Up Node.js

- **Set up a Project Directory:** Create a new project directory and initialize it using

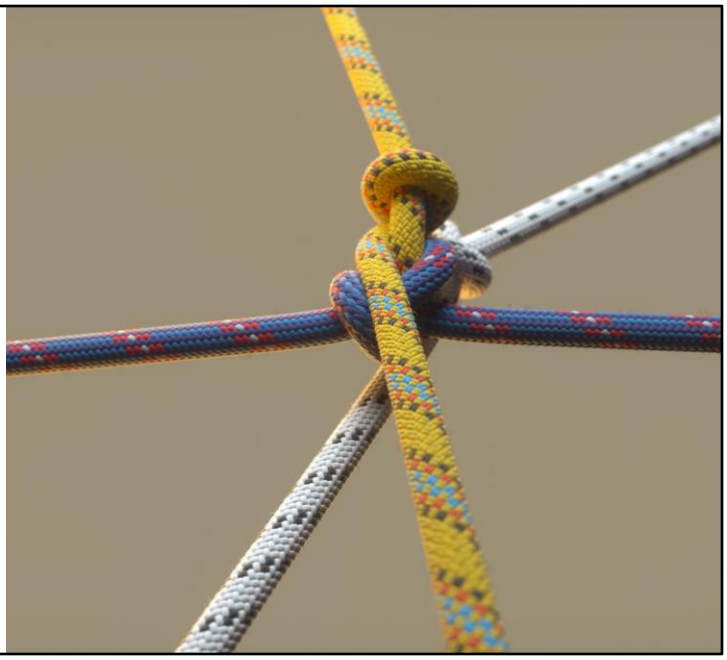
```
mkdir myNodeApp
cd myNodeApp
```

- **Initialize Project:** Initialize a new project using npm (Node Package Manager):

```
npm init -y
```

This command generates a package.json file that contains the project's metadata, dependencies, and configurations.

Creating Your First Node.js Application



Step 1: Create a JavaScript file called app.js

a basic Node.js application that serves an HTTP server.

```
// Load the http module to create an HTTP server
const http = require('http');
// Configure the HTTP server to respond with "Hello, World!" to every
request
const server = http.createServer((req, res) => {
  res.statusCode = 200; // Status code 200 means OK
  res.setHeader('Content-Type', 'text/plain'); // Set content type as plain
text
  res.end('Hello, World!\n'); // Send the response to the client
});
// Listen on port 3000 and IP address 127.0.0.1 (localhost)
server.listen(3000, '127.0.0.1', () => {
  console.log('Server running at http://127.0.0.1:3000/');
});
```

We are creating a basic Node.js application that serves an HTTP server.

http module: This core module allows you to create an HTTP server.
http.createServer(): Creates an HTTP server that listens for requests. The callback function contains the logic to handle incoming requests.
res.statusCode: Sets the HTTP status code for the response (200 indicates success).
res.setHeader(): Sets the response headers, indicating that the content is plain text.
res.end(): Ends the response and sends the data back to the client.
server.listen(): Tells the server to listen for incoming requests on port 3000

Step 2: Run the Server

Run the app using the command:

```
node app.js
```

Now, visit http://127.0.0.1:3000 in your browser, and you should see "Hello, World!" printed on the page.

a basic Node.js application that serves an HTTP server.

What is NPM?

- npm stands for Node Package Manager, and it is the default package manager for the JavaScript runtime environment Node.js.
- npm is an essential tool for managing packages (also known as libraries or modules) that you use in your Node.js projects.
- Package Management
- Command Line Interface (CLI)
- Package Registry
- Versioning and Dependency Management

A package in Node.js contains all the files you need for a module.
Modules are JavaScript libraries you can include in your project.

- Package Management: npm helps you install, update, and manage dependencies (external libraries or modules) that your Node.js application needs to function. It allows developers to access a vast ecosystem of open-source packages.
- Command Line Interface (CLI): npm provides a command-line interface to interact with the registry, install packages, manage project dependencies, and automate scripts.
- Package Registry: The npm registry is an online database that hosts thousands of JavaScript packages, and it's the largest ecosystem of open-source libraries. Developers can upload their own libraries to the registry, making them available for others to use.
- Versioning and Dependency Management: npm allows you to manage the versions of the libraries you are using in your project and ensures that your project has the correct dependencies by using a package.json file.

package.json File

- The package.json file is a fundamental part of every Node.js project. It holds metadata about your project, including the dependencies, scripts, version, author, and more. It is created when you initialize a project using npm init.
- **scripts:** Defines custom commands like npm run start to automate tasks.
- **dependencies:** Lists the libraries your application needs to run.
- **devDependencies:** Lists the libraries used during development, such as testing libraries or build tools.

- **npm** is the default package manager for Node.js, providing a way to install, update, and manage dependencies (libraries or packages) in your project.
- npm has a large ecosystem of packages that can be installed via the npm registry.
- It also supports project management with package.json, enabling you to define dependencies, scripts, and project metadata.
- By using npm, developers can easily manage their JavaScript libraries, automate common tasks, and maintain a consistent environment across different systems and developers.

```
{  "name": "my-node-app",
  "version": "1.0.0",
  "description": "A simple Node.js application",
  "main": "app.js",
  "scripts": {
    "start": "node app.js" },
  "dependencies": {
    "express": "^4.17.1"
  },
  "devDependencies": {
    "nodemon": "^2.0.4"
  },
  "author": "Your Name",
  "license": "MIT"}
```

scripts: Defines custom commands like npm run start to automate tasks.
dependencies: Lists the libraries your application needs to run.
devDependencies: Lists the libraries used during development, such as testing libraries or build tools.

Modules in Node.js

- In Node.js, **modules** are individual files or collections of code that encapsulate specific functionality.
- Modules allow developers to organize their code into smaller, reusable pieces. Modules in Node.js are divided into three types:

1. **Core Modules**: Built-in modules that come with Node.js.
2. **External Modules**: Modules that you install from npm (Node Package Manager).
3. **Custom Modules**: Modules that you write yourself to encapsulate specific functionality for your app.

Core Modules

- Node.js comes with a set of built-in modules that can be used without any additional installation.
- These modules provide basic functionality needed for common tasks such as **handling HTTP reque**sts, working with the **file system**, or **interacting with the operating system**.
- Example 1: http module
  - The first coding example of “hello world”
- Example 2: fs module
  - The fs (File System) module allows you to interact with files on your computer. It can be used to read, write, and manipulate files.

Core modules like http, fs (file system), path, os, and many others are available by default in Node.js.

We use the built-in http module to create a basic web server.
http.createServer() creates a server that listens for requests. When a request is made, it sends a response with the message "Hello, World!".
server.listen() binds the server to a specific port (3000 in this case).

```
const fs = require('fs');

// Write data to a file
fs.writeFile('example.txt', 'Hello, Node.js!', (err) => {
  if (err) throw err;
  console.log('File has been saved!');
});

// Read data from a file
fs.readFile('example.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);  // Outputs: Hello, Node.js!
});
```

Here, the fs.writeFile() method writes data to a file, and fs.readFile() reads the content of the file.

URL Module

- The URL module splits up a web address into readable parts.
- To include the URL module, use the require() method:

```
var url = require('url’);
```

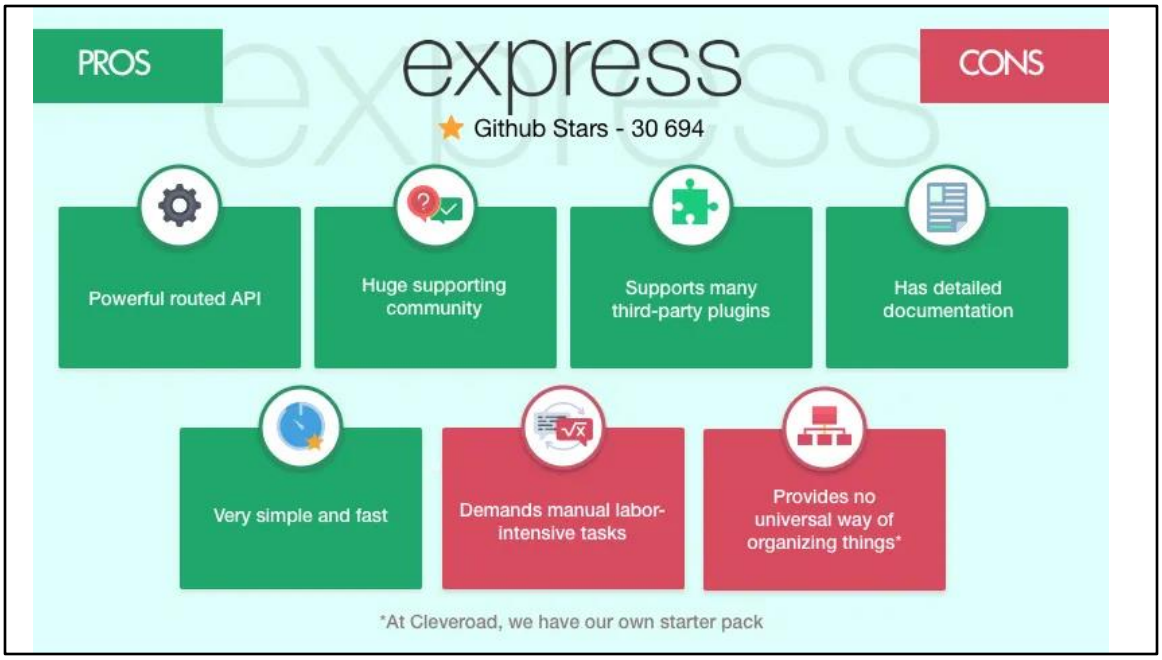
- Parse an address with the url.parse() method, and it will return a URL object with each part of the address as properties:

```
var url = require('url');
var adr = 'http://localhost:8080/default.htm?year=2017&month=february';
var q = url.parse(adr, true);
console.log(q.host); //returns 'localhost:8080'
console.log(q.pathname); //returns '/default.htm'
console.log(q.search); //returns '?year=2017&month=february'
var qdata = q.query; //returns an object: { year: 2017, month: 'february' }
console.log(qdata.month); //returns 'february'
```

- [ ] fs (File System module)
- [ ] path (Directories and paths in node.js)
- [ ] os (Operating System modules)
- [ ] HTTP and HTTPS module
- [ ] util (Formatting strings and debugging)
- [ ] querystring (Parsing and formatting URL query)
- [ ] crypto (Hashing and HMAC, ciphering)
- [ ] events (Emit and listen events)
- [ ] stream (Reading and writing streams of data)
- [ ] zlib (Compression & Decompression)
- [ ] dns (DNS lookup and name resolution functions)
- [ ] buffer (work with binary data and large files)
- [ ] url (Parsing and formatting URLs)
- [ ] net (Network capabilities creating server and clients)
- [ ] tls (Implement Transport and Security layers protocols)
- [ ] child_process (Execution of external commands)
- [ ] process (Process related tasks)
- [ ] readline (reading lines from readable streams)
- [ ] timers (Scheduling and Canceling of code)

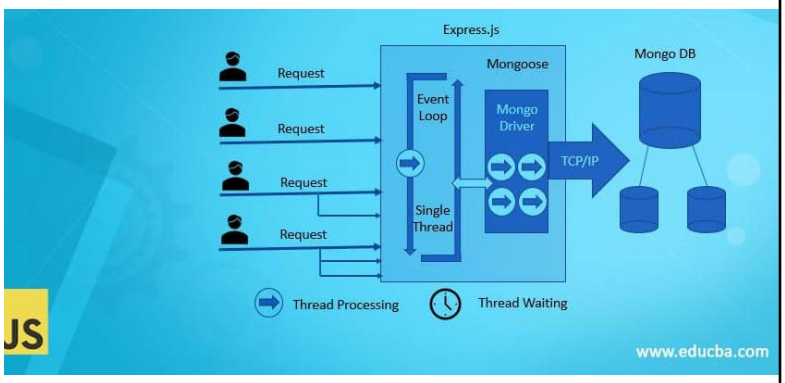
External Modules in Node.js

- External modules are packages that are not included in the core Node.js distribution but can be installed using npm (Node Package Manager).
- These modules extend the functionality of Node.js and provide tools for a wide range of use cases, from working with databases to handling HTTP requests.
- To install external modules, you use the npm install command. You can install the Express.js framework (a popular framework for building web applications) for your project.


PROS
express
CONS
Github Stars - 30 694
Powerful routed API
Huge supporting community
Supports many third-party plugins
Has detailed documentation
Very simple and fast
Demands manual labor-intensive tasks
Provides no universal way of organizing things*
*At Cleveroad, we have our own starter pack

Key Features

- Routing
- Middleware
- Template Engines
- Request Handling
- Static Files
- Error Handling
- Security



Routing:
•Express provides a simple and powerful way to define routes for HTTP requests (GET, POST, PUT, DELETE, etc.) and map them to callback functions.
•Routing can be done with parameters and query strings, making it flexible for different types of web applications.
Middleware:
•Express allows you to use middleware to process requests before reaching the final request handler (route handler). Middleware can perform tasks such as logging, authentication, request parsing, etc.
•Express has built-in middleware (e.g., express.json(), express.urlencoded()) and also allows custom middleware to be added.
Template Engines:
•Express can integrate with various template engines like **EJS**, **Pug**, and **Handlebars** to render dynamic HTML views based on data passed from the server.
Request Handling:
•Express allows handling different HTTP request methods (GET, POST, PUT, DELETE) and offers functionality to retrieve request data (e.g., URL parameters, query strings, form data).

Static Files:
•Express makes it easy to serve static files (such as images, stylesheets, and JavaScript files) through the use of express.static() middleware.
Error Handling:
•Express allows you to define custom error-handling middleware to capture errors and send appropriate responses to the client.
Security:
•Express is often used with additional packages to secure web applications. For example, middleware like helmet can help secure HTTP headers, and cors can help manage cross-origin requests.

How to use Express Framework

- While Node.js is powerful, the http module requires a lot of boilerplate code. To simplify server creation, we use the Express.js framework.
- Express simplifies server creation.
- Install Express using npm

```
npm install express
```

- We will use Express to create a Web Server

```
// Import the external express module
const express = require('express');
const app = express();

// Define a simple route
app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

// Make the server listen on port 3000
app.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

We install Express and import it using require('express').We use app.get() to define a route that responds with "Hello from Express!" when the user visits the homepage.The server listens on port 3000.

Custom Modules

- Custom modules are JavaScript files that you create to organize your code.
- Instead of writing everything in a single file, you can break your application into smaller modules, each responsible for a specific piece of functionality.

Example 1: Greeting

- Create a new JavaScript file (greeting.js):

```
// Export a function that returns a greeting message
module.exports = function(name) {
  return `Hello, ${name}!`;
};
```

- Import the custom module into another file (**custapp.js**):

```
// Import the custom greeting module
const greeting = require('./greeting');

// Use the greeting function
console.log(greeting('World'));  // Outputs: Hello, World!
```

We create a custom module called greeting.js that exports a function.We import the module using require('./greeting') in the main file (app.js) and call the greeting() function to display a message.

Example 2: Calculator

Create a file calculator.js

```
module.exports.add = function(a, b) {
  return a + b;
};
module.exports.subtract = function(a, b) {
  return a - b;
};
module.exports.multiply = function(a, b) {
  return a * b;
};
module.exports.divide = function(a, b) {
  if (b === 0) {
    return 'Cannot divide by zero';
  }
  return a / b;
};
```

Example 2: Calculator

Use the calculator.js module in **calapp.js**:

```
const calculator = require('./calculator');

console.log(calculator.add(5, 3));      // 8
console.log(calculator.subtract(5, 3));  // 2
console.log(calculator.multiply(5, 3));  // 15
console.log(calculator.divide(6, 3));    // 2
```

We create a modular and reusable code structure for our project, separating each piece of functionality into distinct files.

Key Concepts of Working with Modules

require(): This function is used to import built-in modules, external modules, or custom modules into your project. For example:

- `const http = require('http')` — Imports the core http module.
- `const express = require('express')` — Imports the external express module.
- `const greeting = require('./greeting')` — Imports a custom module named greeting.js.

module.exports: This is how we export functionality from a custom module. We can export functions, objects, or variables so that other parts of the application can use them.

Key Concepts of Working with Modules

- **External Dependencies** (node_modules): When we install external modules via npm, they are saved in the node_modules directory in your project. This directory holds all of the project's dependencies.
- By dividing your application into smaller, manageable modules, you can easily maintain and scale your projects.
- Each module should focus on a single responsibility, making your code cleaner and easier to understand.

Building a Full-Stack Application with Node.js and React

Step 1: Initialize the Project...

Create the project directory:

```
mkdir fullstackApp
cd fullstackApp
```

Initialize the backend: Inside the project directory, run:

```
mkdir backend
cd backend
npm init -y
npm install express
```

Step 1: Initialize the Project

Create a simple Express server. Inside backend, create a file called server.js:

```
const express = require('express');
const app = express();
// Enable CORS for the frontend to communicate with the backend
const cors = require('cors');
app.use(cors());
app.get('/api/message', (req, res) => {
  res.json({ message: 'Hello from the backend!' });
});
app.listen(5000, () => {
  console.log('Backend server running on http://localhost:5000');
});
```

This backend server will handle requests from the React frontend.

express: This is the Express framework. It is required at the top of the file to use its features for creating the backend server. Express simplifies routing, middleware, and request handling for Node.js.

app: Here, we are creating an instance of an Express application by calling express(). This app object will be used to define routes and handle HTTP requests.

cors: This line imports the **CORS (Cross-Origin Resource Sharing)** middleware. CORS allows your frontend (running on a different port or domain) to make requests to the backend server. It is essential when your React frontend and Express backend run on different ports (e.g., React runs on port 3000 and Express on port 5000). Without CORS, browsers will block requests from different origins for security reasons.

app.use(cors()): This line tells Express to use the cors middleware, enabling your backend to accept requests from any origin (like your React frontend on

http://localhost:3000). The CORS middleware will automatically handle CORS headers, allowing cross-origin requests.

•**app.get('/api/message', ...)**: This defines a **GET route** on the path /api/message. When a GET request is made to this route, the server responds with a JSON object.

•**req**: The request object, which contains details about the HTTP request (like headers, parameters, etc.).

•**res**: The response object, which is used to send back a response to the client.

•**res.json({ message: 'Hello from the backend!' })**: This sends a JSON response containing a message: "Hello from the backend!". The frontend will be able to read this response and display it.

•**app.listen(5000, ...)**: This starts the server and listens for incoming HTTP requests on port 5000. When the server starts, it logs a message to the console saying that the backend server is running at http://localhost:5000.

Step 2: Create the React Frontend

Create the frontend using create-react-app. In the root directory, run:

```
npx create-react-app frontend
cd frontend
```

Edit the App component. Open src/App.js and modify it as follows

This backend server will handle requests from the React frontend.

```
import React, { useEffect, useState } from 'react';
function App() {
  const [message, setMessage] = useState('');
  // Fetch message from the backend API
  useEffect(() => {
    fetch('http://localhost:5000/api/message')
      .then(response => response.json())
      .then(data => setMessage(data.message));
  }, []);
  return (
    <div className="App">
      <h1>{message}</h1>
    </div>
  );
}
export default App;
```

This React app fetches data from the Node.js backend and displays it.

import React, { useEffect, useState }: This line imports the necessary hooks (useState, useEffect) from React. **useState** is used to manage the state of variables in the component, and **useEffect** allows you to perform side effects, like fetching data from an API.

function App(): This is the main component function for your React app. Inside this function, you define the component's behavior and structure.

const [message, setMessage] = useState('');: Here, the message state is initialized with an empty string. The setMessage function will be used to update the message state when data is fetched from the backend.

useEffect(() => { ... }, []): This is a hook that runs after the component has mounted (i.e., rendered for the first time). The empty dependency array [] ensures the effect runs only once when the component is initially loaded.

fetch('http://localhost:5000/api/message'): This makes a **fetch request** to the

backend server at http://localhost:5000/api/message. The fetch request will get the data from the Express API you defined earlier.

.then(response => response.json()): After receiving the response, the .json() method is used to parse the JSON data sent by the backend.

.then(data => setMessage(data.message)): Once the data is fetched, we update the message state with the value received from the backend (data.message).

return (...): The component returns JSX, which defines the structure of the frontend UI. The content of the h1 tag is dynamically populated with the message state.

<h1>{message}</h1>: This will display the message received from the backend API. Initially, it will be empty (''), but after the data is fetched from the backend, it will show "Hello from the backend!" as defined in the Express server.

export default App;: This exports the App component as the default export, making it available to other parts of the application (e.g., for rendering in index.js).

Step 3: Start both servers

- Start the backend server. In the backend directory, run:

```
node server.js
```

- Start the frontend server. In the frontend directory, run:

```
npm start
```

- Navigate to http://localhost:3000 in your browser, and you should see "Hello from the backend!" displayed, coming from your Node.js API.

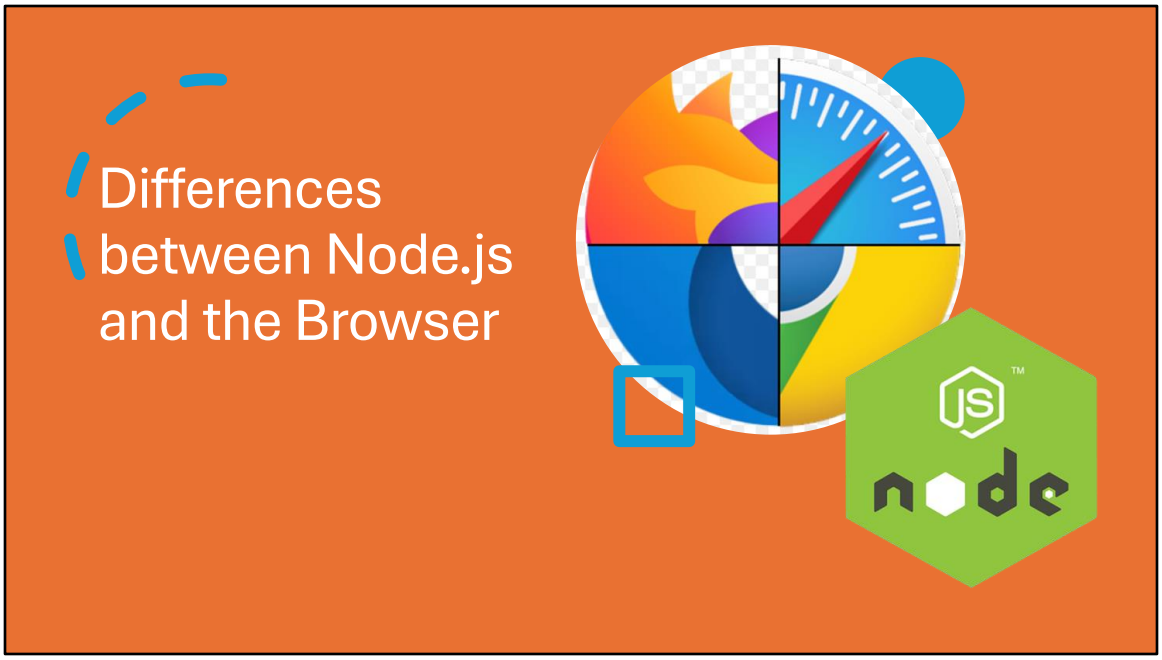
How the Communication Works

- The frontend (React app) makes a GET request to the backend server at http://localhost:5000/api/message.
- The backend server (Express) listens for requests at this endpoint and responds with a JSON object containing a message.
- Once the frontend receives the message, it updates the state and displays it in the h1 tag.

This is a basic **client-server communication pattern** where the frontend fetches data from the backend and displays it dynamically.

Key Take-aways of the Example

- **Backend (Express):** Defines routes, handles requests, sends responses.
- **Frontend (React):** Fetches data from the backend and updates the UI based on the received data.
- **CORS:** Allows communication between the frontend and backend even if they are running on different ports.



Both Browser and Node.js are environments for running JavaScript, but they are designed for different purposes and operate in distinct contexts.

Environment and Context

Browser

- The browser is a client-side environment where JavaScript runs in the user's web browser (like Chrome, Firefox, Safari, etc.).
- Browsers are designed to render and interact with web pages, handling things like user interface (UI) updates, DOM manipulation, and communication with web servers via HTTP requests (using AJAX, Fetch, or WebSockets).
- JavaScript in the browser can access and modify the Document Object Model (DOM), which represents the structure of a web page.

Node.js

- Node.js is a server-side environment that allows you to run JavaScript outside the browser, typically on a server.
- It provides a runtime for executing JavaScript for tasks like building server applications, interacting with databases, file systems, and more.
- Node.js does not have access to the DOM or browser-specific features like HTML rendering but provides its own set of APIs for file system management, networking, and other server-side tasks.

Purpose and Use Cases

Browser

- Client-side operations: Browsers are used for tasks like rendering HTML pages, styling with CSS, and handling user input events (clicks, form submissions).
- Interactivity on web pages: JavaScript in the browser can be used to create dynamic web pages (e.g., updating content without reloading, animations, etc.) using APIs like DOM, CSSOM, and Web Storage.

Node.js

- Server-side operations: Node.js is commonly used for building back-end applications, APIs, web servers, and handling networking tasks.
- Non-blocking I/O: Node.js is known for handling asynchronous operations, such as reading files, making HTTP requests, or querying databases without blocking the main thread.

Purpose and Use Cases

Browser

The browser provides browser-specific APIs like:

- DOM (Document Object Model) for manipulating web page structure.
- Canvas API for drawing graphics and animations.
- Web Storage API (localStorage, sessionStorage) for saving data in the browser.
- Geolocation API for getting the user's location.
- Fetch API for making HTTP requests to servers.

The browser environment focuses on web page rendering, user interaction, and sending data over HTTP.

Node.js

Node.js provides server-side APIs:

- File System (fs) for reading, writing, and manipulating files on the server.
- http module to create web servers and handle HTTP requests and responses.
- path module for handling file and directory paths.
- os module for getting information about the operating system.
- stream module for handling streaming data.

Node.js is designed for building scalable, networked applications, interacting with databases, file systems, and other services.

Event Loop and Execution Model

Browser

- Browsers also use an event-driven, non-blocking I/O model, but their focus is more on UI events (like clicks, keypresses, or mouse movements) and DOM manipulation.
- JavaScript in the browser runs on a single thread, and the browser is responsible for handling interactions with the DOM, and other UI-related tasks.

Node.js

- Node.js runs on a single thread using a non-blocking event-driven model. It uses an event loop to handle asynchronous operations (e.g., database calls, file reads) efficiently without blocking the main thread.
- Node.js makes use of libuv, a C library that provides the event loop and low-level I/O functionalities.

Event Loop and Execution Model

Browser

In the browser, the global object is window. It exposes methods like alert(), setTimeout(), and document for interacting with the DOM.

Other browser-specific objects include:

- document: For interacting with the HTML document.
- localStorage/sessionStorage: For saving data locally on the user's device.
- navigator: For detecting the browser and device.

Node.js

In Node.js, the global object is global. It does not have access to browser-specific objects like window or document.

Node.js exposes several useful global variables, including:

- __dirname: The directory name of the current module.
- __filename: The full filename of the current module.
- process: Provides information and control over the Node.js process (e.g., exit, environment variables).
- require: To import modules (both built-in and custom).

Summary of all differences

- **Browsers** are designed for running JavaScript in the context of interacting with web pages, DOM manipulation, and user interfaces.
- **Node.js** is designed to run JavaScript on the server side, providing tools for building APIs, handling file systems, networking, and much more.

Useful Resources

- https://nodejs.org/en/learn/getting-started/introduction-to-nodejs
- https://www.w3schools.com/nodejs/nodejs_get_started.asp
- https://www.w3schools.com/nodejs/nodejs_email.asp